

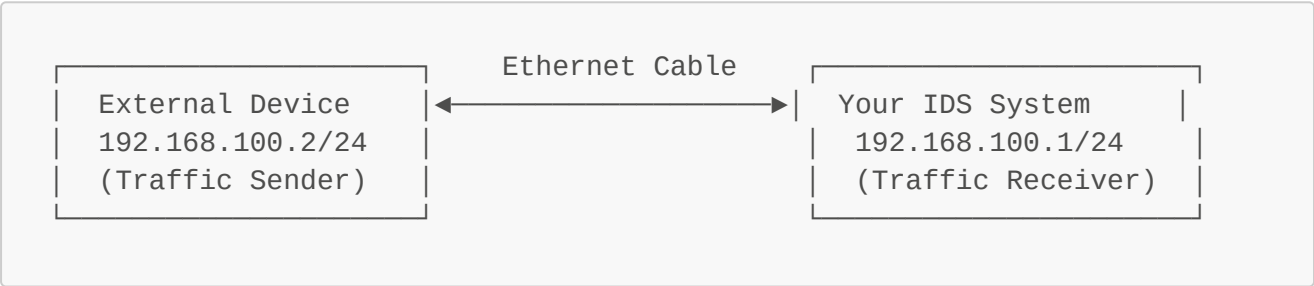
# Complete External Device Setup Guide

This guide will help you configure the **external device** (the computer that will send PCAP traffic to your IDS system).

## What You Need

- **External Device:** Another computer, laptop, Raspberry Pi, or VM
- **Ethernet Cable:** To connect external device to your IDS system's USB adapter
- **OS Options:** Linux (Ubuntu/Kali/Debian), Windows, or macOS

## Quick Setup Summary



## Setup for Linux External Device

Option A: Ubuntu/Debian/Kali Linux

### Step 1: Find Your Ethernet Interface

```
# List all network interfaces
ip link show

# Output will show something like:
# 1: lo: <LOOPBACK,UP,LOWER_UP> ...
# 2: eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> ... ← This is your Ethernet
# 3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> ...

# Common names: eth0, enp0s3, eno1, enp3s0
```

### Step 2: Download the Setup Script

Save this as `setup_external.sh`:

```
#!/bin/bash
# External Device Network Setup Script

# Configuration
INTERFACE="eth0" # ← CHANGE THIS to your interface name from Step 1
IP_ADDRESS="192.168.100.2"
SUBNET="24"
IDS_GATEWAY="192.168.100.1"

echo "
echo "|| External Device Network Configuration ||"
echo "
echo ""
echo "Configuration:"
echo "  Interface: $INTERFACE"
echo "  IP Address: $IP_ADDRESS/$SUBNET"
echo "  Gateway: $IDS_GATEWAY"
echo ""

# Check if interface exists
if ! ip link show "$INTERFACE" &> /dev/null; then
    echo "x Error: Interface $INTERFACE not found!"
    echo "Available interfaces:"
    ip link show | grep '^[0-9]' | awk '{print "  - " $2}' | sed
's/:$//'
    exit 1
fi

echo "Step 1: Stopping NetworkManager interference..."
# Stop NetworkManager from managing this interface
sudo nmcli device set "$INTERFACE" managed no 2>/dev/null || true
sudo systemctl stop NetworkManager 2>/dev/null || true
echo "✓ NetworkManager disabled"

echo ""
echo "Step 2: Flushing existing configuration..."
sudo ip addr flush dev "$INTERFACE"
echo "✓ Configuration flushed"

echo ""
echo "Step 3: Configuring IP address..."
sudo ip addr add ${IP_ADDRESS}/${SUBNET} dev "$INTERFACE"
echo "✓ IP configured: $IP_ADDRESS/$SUBNET"

echo ""
echo "Step 4: Bringing interface up..."
sudo ip link set "$INTERFACE" up
sleep 2
echo "✓ Interface is up"

echo ""
echo "Step 5: Adding route..."
```

```

sudo ip route add 192.168.100.0/24 dev "$INTERFACE" 2>/dev/null || echo
" (Route already exists)"
echo "✓ Route configured"

echo ""
echo "=====
echo "Configuration Complete!"
echo "=====
echo ""

echo "Interface Status:"
ip addr show "$INTERFACE" | grep -E "inet |link/"
echo ""

echo "Routing Table:"
ip route show | grep 192.168.100
echo ""

echo "Testing Connectivity..."
if ping -c 3 -W 2 "$IDS_GATEWAY"; then
    echo ""
    echo "✓ SUCCESS! Connected to IDS system at $IDS_GATEWAY"
    echo ""
    echo "=====
    echo "Next Steps:"
    echo "1. Install tcpreplay: sudo apt install tcpreplay"
    echo "2. Get a PCAP file to replay"
    echo "3. Replay traffic: sudo tcpreplay -i $INTERFACE -K --mbps 10
your_file.pcap"
    echo "=====
else
    echo ""
    echo "⚠ Warning: Cannot ping IDS system at $IDS_GATEWAY"
    echo ""
    echo "Troubleshooting:"
    echo "1. Check Ethernet cable is connected"
    echo "2. Check IDS system is configured (should have run
00_setup_external_capture.sh)"
    echo "3. On IDS system, verify: ip addr show enx00e04c36074c"
    echo "4. Check link status: ethtool $INTERFACE | grep 'Link
detected'"
fi

```

PROF

### Step 3: Run the Setup Script

```

# Make it executable
chmod +x setup_external.sh

# Edit the INTERFACE variable first (change eth0 to your interface)
nano setup_external.sh # Change line: INTERFACE="eth0" to your actual
interface

```

```
# Run it
sudo ./setup_external.sh
```

## Option B: Manual Configuration (if script doesn't work)

```
# Replace eth0 with your actual interface name
INTERFACE="eth0"

# Stop NetworkManager
sudo nmcli device set $INTERFACE managed no

# Configure the interface
sudo ip addr flush dev $INTERFACE
sudo ip addr add 192.168.100.2/24 dev $INTERFACE
sudo ip link set $INTERFACE up

# Verify
ip addr show $INTERFACE
ip route show | grep 192.168.100

# Test
ping 192.168.100.1
```

---

## 📦 Setup for Windows External Device

### Step 1: Open PowerShell as Administrator

- Press **Win + X**
- Select "Windows PowerShell (Admin)" or "Terminal (Admin)"

### Step 2: Find Your Ethernet Interface

```
Get-NetAdapter | Where-Object {$_.MediaType -eq "802.3"}
```

### Step 3: Configure IP Address

```
# Replace "Ethernet" with your actual adapter name from Step 2
$InterfaceName = "Ethernet"

# Remove existing IP configuration
Remove-NetIPAddress -InterfaceAlias $InterfaceName -Confirm:$false -
ErrorAction SilentlyContinue
Remove-NetRoute -InterfaceAlias $InterfaceName -Confirm:$false -
ErrorAction SilentlyContinue
```

```
# Configure new IP
New-NetIPAddress -InterfaceAlias $InterfaceName -IPAddress 192.168.100.2
-PrefixLength 24 -DefaultGateway 192.168.100.1

# Verify
Get-NetIPAddress -InterfaceAlias $InterfaceName
```

#### Step 4: Test Connectivity

```
ping 192.168.100.1
```

#### Step 5: Install tcpreplay (for PCAP replay)

- Download and install Npcap: <https://npcap.com/>
- Install tcpreplay via Cygwin or WSL2

---

## Setup for macOS External Device

#### Step 1: Find Your Ethernet Interface

```
ifconfig | grep "^en" | cut -d: -f1

# Common names: en0 (usually WiFi), en1 or en2 (Ethernet)
# Or use: networksetup -listallhardwareports
```

#### Step 2: Configure IP Address

```
# Replace en1 with your Ethernet interface
INTERFACE="en1"

# Configure IP
sudo ifconfig $INTERFACE inet 192.168.100.2 netmask 255.255.255.0 up

# Add route
sudo route -n add 192.168.100.0/24 -interface $INTERFACE

# Verify
ifconfig $INTERFACE
netstat -rn | grep 192.168.100
```

#### Step 3: Test Connectivity

```
ping 192.168.100.1
```

---

## After Configuration - Traffic Generation

Once your external device is configured and can ping 192.168.100.1:

Install tcpreplay

**Ubuntu/Debian/Kali:**

```
sudo apt update
sudo apt install tcpreplay
```

**CentOS/RHEL:**

```
sudo yum install tcpreplay
```

**macOS:**

```
brew install tcpreplay
```

Get PCAP Files

**Option 1: Use Your Project's PCAPs**

```
# If you have PCAPs on your IDS system, transfer them
# On IDS system:
cd ~/Programming/IDS/pcap_samples
ls *.pcap

# Copy to external device (from IDS system)
scp *.pcap user@external-device-ip:/tmp/
```

**Option 2: Download Sample Attack PCAPs**

```
# Download sample attack traffic
wget https://www.malware-traffic-analysis.net/2023/12/01/2023-12-01-traffic-analysis-exercise.pcap.zip
unzip 2023-12-01-traffic-analysis-exercise.pcap.zip
```

---

### Option 3: Create Simple Test Traffic

```
# Generate HTTP traffic with tcpdump (on IDS system first)
sudo tcpdump -i any -w test.pcap -c 100 port 80
# Then transfer to external device
```

### Replay PCAP Traffic

```
# Replace eth0 with your interface name
INTERFACE="eth0"

# Basic replay
sudo tcpreplay -i $INTERFACE your_file.pcap

# Replay with speed control (10 Mbps)
sudo tcpreplay -i $INTERFACE -K --mbps 10 your_file.pcap

# Replay as fast as possible
sudo tcpreplay -i $INTERFACE -t your_file.pcap

# Loop replay 10 times
sudo tcpreplay -i $INTERFACE --loop 10 your_file.pcap

# Edit destination MAC to match IDS USB adapter
sudo tcpreplay -i $INTERFACE --enet-dmac=00:e0:4c:36:07:4c
your_file.pcap
```

---

## 🔍 Verification Checklist

PROF

On **External Device**, verify:

```
# 1. Interface is UP
ip link show eth0 # Should show "state UP"

# 2. IP is configured
ip addr show eth0 # Should show "inet 192.168.100.2/24"

# 3. Route exists
ip route show | grep 192.168.100 # Should show route via eth0

# 4. Cable connected
ethtool eth0 | grep "Link detected" # Should show "yes"

# 5. Can ping IDS
ping 192.168.100.1 # Should get replies
```

```
# 6. ARP resolved
arp -a | grep 192.168.100.1 # Should show MAC address
```

On **IDS System**, verify:

```
# Check interface configured
ip addr show enx00e04c36074c # Should show "inet 192.168.100.1/24"

# Can ping external device
ping 192.168.100.2

# Monitor for incoming traffic
sudo tcpdump -i enx00e04c36074c -n
```



## Troubleshooting

Problem: "Network is unreachable"

**Solution:**

```
# On external device, add route explicitly
sudo ip route add 192.168.100.0/24 dev eth0

# Or delete and recreate with proper subnet
sudo ip addr del 192.168.100.2/24 dev eth0
sudo ip addr add 192.168.100.2/24 dev eth0
```

Problem: "Next hop has invalid gateway"

PROF

**Solution:**

- Don't add a default route for direct connection
- The `/24` subnet mask automatically creates the route
- Only configure: `sudo ip addr add 192.168.100.2/24 dev eth0`

Problem: "Cannot ping IDS system"

**Check on External Device:**

```
# Interface status
ip link show eth0 # Must show "state UP"

# IP configuration
ip addr show eth0 # Must show "inet 192.168.100.2/24"
```



```
# Routing
ip route show # Must show "192.168.100.0/24 dev eth0"

# Physical link
ethtool eth0 | grep "Link detected" # Must show "yes"
```

### Check on IDS System:

```
# Interface status
ip addr show enx00e04c36074c # Must show "inet 192.168.100.1/24"

# Promiscuous mode
ip link show enx00e04c36074c | grep PROMISC # Must show PROMISC flag
```

Problem: "Permission denied" or "Operation not permitted"

### Solution:

```
# Use sudo for all network commands
sudo ip addr add 192.168.100.2/24 dev eth0
sudo ip link set eth0 up
sudo tcpreplay -i eth0 file.pcap
```

Problem: NetworkManager keeps resetting configuration

### Solution:

```
# Permanently disable NetworkManager for this interface
sudo nmcli device set eth0 managed no

# Or edit NetworkManager config
sudo nano /etc/NetworkManager/NetworkManager.conf
# Add:
# [keyfile]
# unmanaged-devices=interface-name:eth0

# Restart NetworkManager
sudo systemctl restart NetworkManager
```

---

## Complete Workflow Example

On External Device (Attack Generator):

---

```
# 1. Setup network
sudo ip addr add 192.168.100.2/24 dev eth0
sudo ip link set eth0 up

# 2. Test connectivity
ping 192.168.100.1

# 3. Install tcpreplay (if not installed)
sudo apt install tcpreplay

# 4. Get a PCAP file
ls *.pcap

# 5. Replay traffic to IDS
sudo tcpreplay -i eth0 -K --mbps 10 attack_traffic.pcap
```

On IDS System (Your Laptop):

```
# Terminal 1: Monitor incoming traffic
sudo tcpdump -i enx00e04c36074c -n

# Terminal 2: Watch Suricata alerts
tail -f dpdk_suricata_ml_pipeline/logs/suricata/eve.json | jq
'select(.event_type=="alert")'

# Terminal 3: Watch ML predictions
tail -f dpdk_suricata_ml_pipeline/logs/ml/consumer.log
```

## Quick Reference

| Setting    | External Device                            | IDS System                          |
|------------|--------------------------------------------|-------------------------------------|
| IP Address | 192.168.100.2/24                           | 192.168.100.1/24                    |
| Interface  | eth0 (or your Ethernet)                    | enx00e04c36074c                     |
| Command    | sudo ip addr add 192.168.100.2/24 dev eth0 | sudo ./00_setup_external_capture.sh |
| Test       | ping 192.168.100.1                         | ping 192.168.100.2                  |
| Traffic    | sudo tcpreplay -i eth0 file.pcap           | sudo tcpdump -i enx00e04c36074c     |

## Success Indicators

You'll know it's working when:

1.  External device can ping 192.168.100.1
  2.  IDS system can ping 192.168.100.2
  3.  `tcpdump` on IDS shows packets when replaying PCAP
  4.  Suricata generates alerts in eve.json
  5.  ML consumer shows predictions in logs
- 

## You're Ready!

Your external device is now configured to send traffic to your IDS system. Start with a simple PCAP replay and watch your IDS detect the attacks!

### Next Steps:

1. Transfer a PCAP file to external device
2. Ensure IDS pipeline is running (`sudo ./quick_start.sh`)
3. Replay the PCAP: `sudo tcpreplay -i eth0 your_file.pcap`
4. Watch the detections happen in real-time! 